real jarkom

Tanpa kamu sadari, sebentar lagi semester 2 akan berakhir :pensive: dan kamu akan lanjut ke semester 3. Konon di semester 3 ada mata kuliah yang sangat amat memberikan kenangan indah yaitu Komunikasi Data dan **Jaringan Komputer** (soal ini bisa dibilang bertemakan jarkom).

Kamu diberikan sebuah jaringan yang terdiri dari **n** node (1-indexed). Terdapat sebuah list **ping**, dengan jumlah elemen **m**, yang berisi waktu perjalanan directed edge, **ping[i] = (a_i, b_i, c_i)** dimana **a_i** merupakan node awal, **b_i** merupakan node tujuan, dan **c_i** merupakan waktu yang dibutuhkan dari node awal ke node tujuan.

Sebuah sinyal akan dikirim dari suatu node **k**. Setiap node juga memiliki suatu **delay**, dimana misal sinyal sampai ke satu node, akan ada waktu tambahan **delay[i]** sebelum node itu dapat mengirimkan sinyal ke node berikutnya. Print waktu minimum dimana semua node n menerima sinyal. Misal tidak mungkin, print -1.

Input Format

Baris pertama berisi **n** dan **k**.
Baris kedua berisi jumlah ping **m**.
Baris ketiga sampai baris ke 3 + m-1 berisi isi list **ping[]**.
Baris terakhir atau 3 + m berisi **delay[]**.

Constraints

- 1 <= k <= n <= 100

- 1 <= ping.length <= 6000
- ping[i].length == 3
- 0 <= delay.length <= 6000
- delay[i].length == n
- 1 <= ai, bi <= n
- ai != bi
- 0 <= ci <= 100
- Semua pair (ai, ci) unik - tidak ada edge yang duplikat.

Output Format

Waktu minimum dimana semua node **n** menerima sinyal atau -1.

Sample Input 0

```
4 2
3
2 1 1
2 3 1
3 4 1
0 1 2 1
```

Sample Output 0

```
5
```

Sample Input 1

```
2 1
1
1 2 1
1 0
```

Sample Output 1

```
2
```

Sample Input 2

```
2 2
1
1 2 1
1 0
```

Sample Output 2

```
-1
```

Code Stub / Template

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <limits>
#include <algorithm>

using namespace std;

class NetworkGraph {
    // ...

public:
    // constructor...

    void addEdge(int u, int v, int weight) {
        // directed graph...
    }
```

```
    void setDelays(const vector<int>& nodeDelays) {
        // ...
    }

    int networkDelayTime(int startNode) {
        // ...
    }
};

int main() {
    // ...

    return 0;
}
```

“Mao Mao dan Kuil”

Deskripsi

Dahulu kala, di Kerajaan Negeri Timur, hiduplah seorang gadis apoteker bernama Mao Mao. Karena kepintaran dan kecerdasannya, ia diminta oleh raja untuk menemani sebuah perjalanan penting menuju kuil mistis yang penuh rahasia.

Konon, di dalam kuil tersebut tersembunyi jawaban atas pertanyaan besar kerajaan:

“Apakah putra mahkota benar-benar pantas menjadi raja?”

Namun, kuil itu bukanlah tempat biasa. Di dalamnya terdapat banyak ruangan yang saling terhubung melalui berbagai pintu. Setiap ruangan bisa saja memiliki satu, dua, atau bahkan banyak pintu menuju ruangan lainnya. Sebagian jalur akan membawa mereka lebih dalam, sementara yang lain hanyalah jalan buntu.

Satu-satunya cara untuk menjawab pertanyaan itu adalah dengan mencapai balkon kuil. Jika mereka sampai ke balkon, maka sang putra mahkota terbukti layak naik takhta.

Mao Mao pun menyarankan untuk menggunakan metode pencarian yang baru saja ia pelajari, yaitu Depth-First Search (DFS), untuk menyusuri kuil dan menemukan jalan yang benar.

Input Format

- Baris pertama berisi tiga bilangan bulat `N`, `S`, dan `B`, yang masing-masing menyatakan jumlah ruangan dalam kuil, ruangan awal tempat raja memulai perjalanan, dan ruangan balkon sebagai tujuan akhir.
- Kemudian terdapat N baris berikutnya.
  - Setiap baris `ke-i` (1 ≤ `i` ≤ `N`) berisi daftar ruangan yang dapat langsung dicapai dari ruangan `ke-i`, diakhiri dengan angka 0.
  - Jika suatu ruangan tidak memiliki pintu keluar, maka baris tersebut hanya berisi angka 0.

Constraints

- 1 ≤ `N` ≤ 10^3
- `S` = 1
- 1 ≤ `B` ≤ `N`

Output Format

- Baris pertama berisi sebuah bilangan bulat yang menunjukkan jumlah jalur eksplorasi yang berakhir di jalan buntu sebelum mencapai balkon.
- Baris kedua berisi daftar bilangan bulat yang merepresentasikan jalur pertama menuju balkon, dimulai dari ruangan awal hingga ruangan balkon. Setiap bilangan dipisahkan dengan spasi.

Contoh Input:

```
6 1 6
2 3 0
4 0
5 6 0
0
0
0
```

Contoh Output:

```
2
1 3 6
```

Penjelasan

DFS pertama: 1 → 2 → 4 → (buntu) → hitung 1.
DFS kedua: 1 → 3 → 5 → (buntu) → hitung 2.
DFS ketiga: 1 → 3 → 6 → berhasil → berhenti.

Kode Soal

```cpp
#include <iostream>
#include <vector>

using namespace std;

class Kuil {
private:
    int N, B, buntuCount = 0;
    vector<vector<int>> adj;
    vector<int> finalPath;
    bool foundBalkon = false;

public:
    Kuil(int n, int b) : N(n), B(b), adj(n + 1) {}

    void addDoors(int ruangan, const vector<int>& pintu) {
        // add your code here
    }

    void dfs(int current, vector<int>& path, vector<bool>& visited) {
        if (foundBalkon) return;

        // add your code here
    }

    void findPath(int start) {
        // add your code here
    }

    void printResult() const {
        // add your code here
    }

};

int main() {
    int N, S, B;
    cin >> N >> S >> B;

    Kuil kuil(N, B);

    // add your code here

    return 0;
}
```

Lantai ujian

Setelah dengan mudah melewati ujian pertama, yaitu Bertahan hidup. Urek mazino beserta para Regular yang tersisa dikirim ke arena ujian berikutnya. Kini, mereka akan menghadapi tantangan yang diberikan oleh Gisley. Bantulah urek mazino melewati ujian ini!

Gisley memberikannya sebuah grid berukuran n×m yang berisi huruf-huruf. Dia mendefinisikan jarak antara dua baris dalam grid sebagai perbedaan absolut terbesar antara huruf-huruf di kolom yang sama.

Urek mazino ditugaskan untuk menandai semua baris. Biaya menandai baris pertama yang dia pilih adalah nol. Setelah itu, biaya menandai setiap baris sama dengan jarak dari baris (yang sedang ditandai) ke salah satu dari baris yang sudah ditandai sebelumnya.

Bantulah Urek untuk menentukan nilai terkecil dari biaya terbesar untuk menandai sebuah baris.

Input Format

- Baris pertama berisi dimensi grid n dan m
- n baris berikutnya berisi string dengan m karakter masing-masing

Constraints

- $2 \le n \le 2000$
- $1 \le m \le 25$

Output Format

nilai terkecil

Sample Input 0

```
3 3
kye
uif
xor
```

Sample Output 0

```
13
```

Code snippet

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>
#include <string>
#include <algorithm>

using namespace std;

class Graph {
private:
    int numVertices;
    vector<vector<pair<int, int>>> adjList;

public:
    Graph(int vertices) : numVertices(vertices), adjList(vertices) {}

    void addEdge(int src, int dest, int weight) {

    }
```

```
    int primMST() {

    }
};

int calculateDistance(const string& row1, const string& row2) {

}

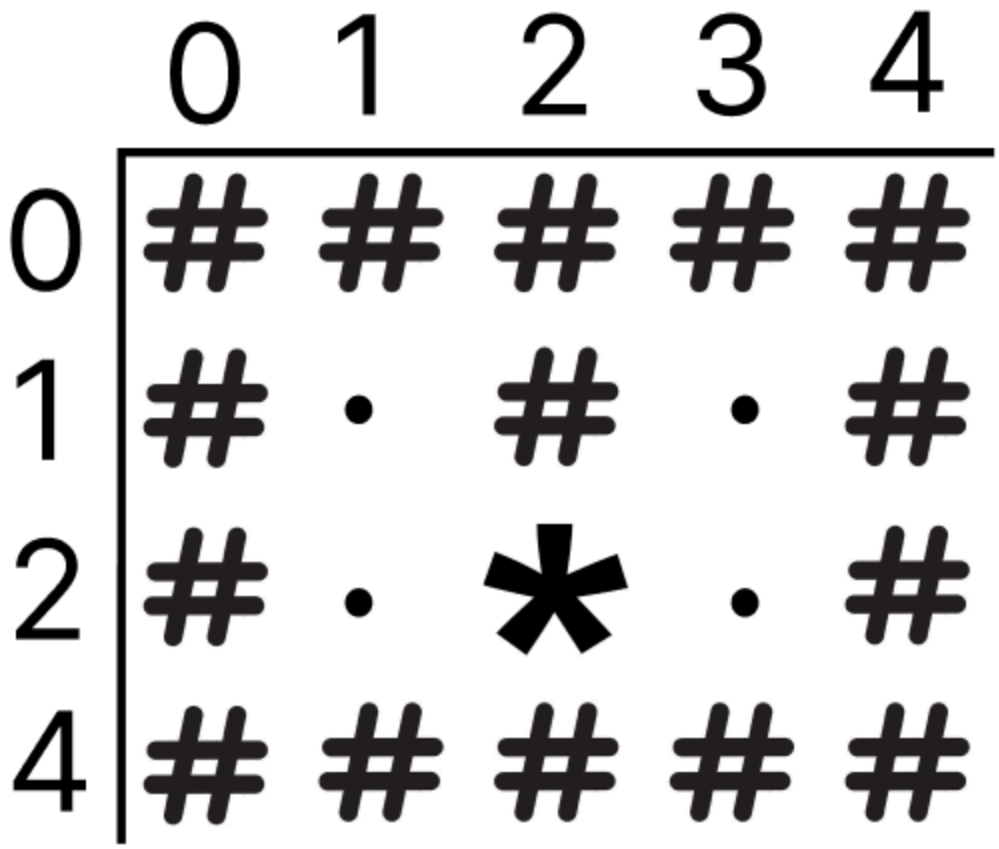
int main() {
    int n, m;
    cin >> n >> m;

    return 0;
}
```

Persiapan Melawan Tenryubito

Lupi adalah seorang bajak laut jadi-jadian yang terbuat dari karet. dia sedang mencari sebanyak mungkin Senjata Kuno asli Palembang untuk melawan ras yang sudah berkuasa lama yaitu, Tenryubito. Berdasarkan petunjuk poneglyph yang ditemukan Nami, **Senjata Kuno asli Palembang berada di lokasi . atau * yang dikelilingi oleh tepat 3 karakter # pada empat arah mata angin (atas, bawah, kiri, kanan).**

Bantu lupi mencari jumlah dan tempat Senjata Kuno asli Palembang untuk melawan Tenryubito. Gunakan Breadth First Search agar lebih mudah



Input Format

- Baris pertama adalah X Y dimana X dan Y adalah integer dimana X adalah kolom dan Y adalah baris
- Baris seterusnya adalah grid sepanjang X diulang Y kali yang menggambarkan grid peta
- '.' menggambarkan ruang kosong
- '#' menggambarkan dinding
- '*' menggambarkan orang

Constraints

- 1 ≤ R, C ≤ 100 (Grid tidak akan lebih besar dari 100x100).
- Grid hanya akan berisi karakter #, ., dan *.
- Akan ada tepat satu karakter * di dalam grid.

Output Format

- Cetak satu baris pertama yang berisi satu bilangan bulat: jumlah total senjata yang dapat diakses oleh orang tersebut.
- Jika jumlahnya lebih dari nol, cetak baris-baris berikutnya. Setiap baris berisi dua bilangan bulat X dan Y (dipisahkan oleh spasi), yang merupakan koordinat baris dan kolom (berbasis nol atau 0-indexed) dari setiap harta karun yang dapat diakses.
- Koordinat harus dicetak dalam urutan terurut. Urutkan berdasarkan kolom (X) terlebih dahulu, kemudian berdasarkan baris (Y).
- Jika tidak ada harta karun yang dapat diakses, hanya cetak angka 0 di baris pertama dan tidak ada baris lain setelahnya.

Sample Input 0

```
5 4
#####
#.#.#
#.*.#
#####
```

Sample Output 0

```
2
1 1
```

```
3 1
```

Code snippet

```
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <set>
#include <utility>
#include <algorithm>

using namespace std;


class TreasureFinder {
    bool isWeapon(int r, int c) {
 // <-your code goes here->
    }

    void findWeaponAndStart() {
 // <-your code goes here->
    }

    void bfs() {
  // <-your code goes here->
       int dr[] = {-1, 1, 0, 0};
       int dc[] = {0, 0, -1, 1};
  // <-your code goes here->
    }

    void outputResult() {
   // <-your code goes here->
    }
};
```

```
int main() {
   // <-your code goes here->
}
```

Kavi bulol

Kavi dan Vika tuh couple goals banget. Tapi sekarang mereka lagi LDR, bukan beda kota... tapi kejebak di dunia bucin 🥲 Satu-satunya cara biar mereka bisa nyambung lagi adalah ngikutin jejak rasa yang tersebar di jaringan misterius. Nah, jejak rasa itu kayak titik-titik yang nyambungin cerita cinta mereka.

Tugas kamu sebagai orang yang melihat Kavi adalah bantuin dia ngelacak jejak rasa itu dari titik awal yang di tentukan sesuai awal mula perasaan kavi menggema (node X).

Ada dua cara buat nyusurin jejak rasa ini:

- Jalur BFS (Breadth-First Search) Ini kayak jalan resmi. Kavi nyusurin satu-satu, dari yang paling deket dulu.
- Jalur DFS (Depth-First Search) Ini kayak kenangan cinta yang dalam, Kavi bisa lompat ke jalur mana aja yang ada di depan.

Setiap node punya angka sinyal. Dan buat dapetin kode buka hati Vika, Kavi harus ngitung kayak gini:

`angka1 * angka2 / angka3 * angka 4 / angka5 …`

Input Format

V → jumlah simpul (node)

N → jumlah jejak rasa (edge)

A B → N baris, sinyal dari node A ke B (arahnya searah, ya)

X → titik awal Kavi mulai cari jejak

Constraints

1. Mulai dari node X.
2. Hasil traversal BFS dan DFS dijadikan urutan angka.
3. Hitung kayak ini:
    - x1 * x2 / x3 * x4 / x5 ... (selang-seling perkalian dan pembagian).
    - Output dibuletin hanya sampai 2 angka di belakang koma.

Output Format

hasil kode dari BFS traversal

hasil kode dari DFS traversal

Sample Input 0

```
4
4
1 2
1 3
2 4
3 4
1
```

Sample Output 0

```
2.67
```

```
1.50
```

Sample Input 1

```
3
2
1 2
2 3
1
```

Sample Output 1

```
0.67
0.67
```